

Pricing Deception: Deciding When a Web Attack Detector Should Probe

Parth Kansal, Amrit Singh Nijjar, and Siddhant Mehta

Department of AIT-CSE, Chandigarh University,
Gharuan, Mohali, Punjab, India

parth.kansal823@gmail.com, amrit.nijj814@gmail.com, 23bis70162@cuchd.in

Abstract. A web application firewall must allow or block each request using only what that request shows. Deception is normally used later, after a session has already been judged hostile. We use it earlier instead. When our system is unsure about a visitor, it adds a small invisible probe to the response. A normal browser never shows the probe. A client that acts on it reveals itself. The system decides when to probe by working out what the answer is worth, not by a threshold we picked by hand. Under ordinary cost accounting there is no middle option at all, so the probe cannot be tuned into existence. Over 99 runs, probing caused an extra 7.0% of attackers to be caught, and no legitimate session was ever diverted.

Keywords: Cyber deception · Web application security · Value of information · Cost-sensitive detection · Honeytokens.

1 Introduction

A web application firewall has to make a choice on every request. It can let the request through, or it can block it. It must choose using only what that one request contains.

This creates a hard trade-off. If the firewall acts early, on weak evidence, it blocks real users. If it waits for strong evidence, the attacker has already sent several requests. Machine-learned detectors soften this problem but do not remove it. They still watch, wait, and then commit to a decision.

Some attacks make the problem worse. A request for `/records/5` looks exactly like a request for `/records/6`. If the second record belongs to someone else, the request is an attack. Nothing in the request says so. There is no pattern to match, so a signature firewall is blind to it. A learned detector also struggles, because an honest reporting tool reads records in order too, and produces the same pattern.

Deception is the usual answer to this kind of blindness. Honey pots and honeypots give the defender something to do besides wait. But in almost every system, deception happens *after* the decision. A session is judged hostile first, and only then is it moved into a honeypot. Deception is a result of detection, not a tool for it.

This paper uses deception as a tool for detection. When our system is unsure about a visitor, it adds something to the response that a browser never displays, but that anyone reading the raw response will see. It may be a fake database error naming a table that does not exist, or an unused field in a JSON reply. An honest user sees nothing. Someone testing the site for weaknesses acts on it, and by acting reveals what a passive score could only wait for. We call these additions *probes*.

The question we answer is not how to build a probe. That is engineering. The question is *when* the detector should use one.

Our answer is that this decision can be calculated. A probe gives no immediate protection, because the request still reaches the real application. Its only value is the information it may reveal. That value can be measured. When we price the probe by the value of the information it buys, probing becomes the cheapest action over a band of belief. The edges of that band are calculated from a fixed cost table and a measured probe success rate. They are not chosen by us.

Contributions. This paper makes two main contributions.

1. **A priced third action.** We show that probing is the cheapest action over a band of belief whose edges are calculated, not tuned. Under ordinary cost accounting this band is empty. There is therefore no middle option to tune, and the probe only appears once the value of information is included.
2. **A measured causal effect.** We do not simply compare a system with probes against one without. Inside the system that probes, we withhold the probe from a random one in ten sessions. Both groups sit at the same belief. The gap between them measures what the probe itself caused.

2 Literature Review

We group earlier work into four areas and state what each leaves open.

Detecting web attacks. Signature firewalls match known bad patterns. They are easy to defeat by obfuscation, and blind to attacks that use valid syntax. Learned detectors replace fixed patterns with a model. Kruegel and Vigna [15] built an early anomaly detector for web requests, and Robertson et al. [19] extended it to generalise across sites. More recent work uses deep models, such as Tekerek [23], and adaptive methods, such as Amouei et al. [2]. All of these share one habit. They watch a session gather evidence and then commit to a score. Our detector belongs to this family and is not our contribution. We use a signature firewall and a passive detector as our two baselines, because our contribution sits on top of them.

Telling scripts from people. A separate line of work asks whether a client is automated, not whether it is hostile. Iliou et al. [10] show that modern bots copy

browser behaviour closely enough that automation signals alone stop separating them from people. This is why our detector scores automation and hostility separately. A scanner is automated and hostile. A price-comparison bot is automated and harmless. A careful human attacker is neither automated nor harmless. One score cannot express the middle case.

Deception and honeytokens. Deception as a defence is well studied. Provos [18] built an early framework for virtual honeypots. Almeshekah and Spafford [1] argue that deception must be planned around a target belief. Pawlick et al. [16] give a taxonomy, and Zhu et al. [27] survey the game-theoretic and learning approaches. Ferguson-Walter et al. [8] study whether decoys actually change what attackers do. In nearly all of this work, the deception is a *place*. A caught attacker is moved there, after the decision has been made.

Planting a fake file or password as a tripwire is also old and widely used. Yuill et al. [26] introduced honeyfiles, Bowen et al. [5] set out the properties a good decoy document needs, and Juels and Rivest [11] introduced fake passwords. That work treats such traps as always on. Kahlhofer and Rass [14] review deception placed inside the application itself, and Kahlhofer et al. [13] automate deploying and rotating these traps. Their work stops where ours starts. It does not decide *when* to place a trap based on what the system currently believes about the visitor. Traps are also not free to hide: Vetterl and Clayton [25] fingerprinted honeypots at scale, and Srinivasa et al. [22] did the same for honeytokens.

Generated fake environments. Language models are increasingly used to produce convincing fake systems. Sladić et al. [20] generate shell output live, and Bridges et al. [6] review the area and find the gains so far are modest. Kahlhofer et al. [12] measure how tempting a planted item looks. What this work measures is realism. Vero et al. [24] propose an evaluation framework covering stealth and fidelity, but not whether a fake system contradicts itself during a long visit. Our fake site is built so that its content generator can be swapped, and we test that separately.

Decision theory. The mathematics we use is old and standard. Howard [9] defined the value of sample information. Elkan [7] set out cost-sensitive decisions under a loss matrix. Pawlick et al. [17] model deception as a signalling game with evidence, and Axelsson [4] showed how the rarity of attacks constrains any intrusion detector. None of this work includes an action that buys evidence. That is the gap we fill. We claim no new mathematics. Our contribution is noticing that a probe *is* a purchase of information, and that pricing it as one turns a hand-set option into a calculated one. We follow the evaluation warnings given by Sommer and Paxson [21] and by Arp et al. [3] throughout.

3 Methodology

3.1 System Overview

Our system is a reverse proxy. It sits in front of the real web application. Clients only ever talk to the proxy. The application never sees a request that has not passed through it, and never knows the deception exists.

Every request follows the same five steps.

1. Identify the session, using a cookie.
2. Turn the request into 18 numbers, called features.
3. Update two running scores for the session.
4. Price the three possible actions and take the cheapest.
5. Write the decision to a log that cannot be edited afterwards.

The three actions are *pass*, *probe* and *divert*. Pass sends the request to the real application. Divert moves the session into a fake copy of the site. Probe adds an invisible probe to the response and passes the request on unchanged.

3.2 Two Scores, Not One

The 18 features split into two groups, because the threat has two parts.

Ten *automation* features describe how the client behaves. They include the time between requests, how regular that timing is, whether images and stylesheets were fetched, how complete the browser headers are, and whether a cookie was sent.

Eight *hostility* features describe what the client is trying to do. They include how many unusual characters appear in user input, whether database keywords appear in a suspicious position, how many replies were errors, how many logins failed, and how many different accounts the session tried to log in to.

Two logistic models turn these features into two scores. Both scores build up across the session. We keep them apart for a reason. The cost of an action depends only on hostility, because an automated client may be harmless. So hostility drives the decision. Automation is used only to choose *which* probe to use, because a script and a human take different bait.

3.3 Choosing an Action by Price

At any moment the system holds a belief p that the visitor is hostile. Each action has a cost, and those costs are fixed and locked before any data is collected. Table 1 gives them.

Three numbers in Table 1 carry the argument. Diverting a harmless visitor is the worst outcome, so it costs 200. Probing a hostile visitor costs the same as passing one, both 25, because the request still reaches the real application either way. The only extra cost of probing is 1, paid by harmless visitors.

Table 1. The fixed cost table. These numbers were locked and hashed before any data was collected, so no threshold below could be adjusted to improve results.

	pass	probe	divert
visitor is harmless	0	1	200
visitor is hostile	25	25	-20

Using these costs, the expected cost of each action is a straight line in p :

$$\mathbb{E}[C_{\text{pass}}] = 25p, \quad \mathbb{E}[C_{\text{probe}}] = 1 + 24p, \quad \mathbb{E}[C_{\text{divert}}] = 200 - 220p. \quad (1)$$

Now compare the first two lines of Equation (1):

$$\mathbb{E}[C_{\text{probe}}] - \mathbb{E}[C_{\text{pass}}] = 1 - p \geq 0. \quad (2)$$

Equation (2) is never negative. Probing always costs more than passing. So on cost alone the system would never probe. It would only pass or divert, and switch between them where $25p = 200 - 220p$, that is at $p \approx 0.816$.

This matters more than it may appear. It means there is no middle option to tune. A reader may reasonably suspect that our probing band is just a threshold we chose. It is not, because without the next step there is no band at all.

3.4 Pricing the Information

A probe is worth something only because of what it may tell us. It produces one of two outcomes: the client takes the bait, or it does not. Either way the belief becomes sharper, and the next decision becomes better.

We measure this using the expected value of sample information [9]:

$$V(p) = \min_a \mathbb{E}[C(a) \mid p] - \mathbb{E}_Z \left[\min_a \mathbb{E}[C(a) \mid p'(Z)] \right]. \quad (3)$$

In Equation (3), Z is what we observe, and $p'(Z)$ is the updated belief after seeing it. Let β_a be the chance a hostile client takes the bait, and β_b the chance a harmless one does. Then the chance of a bite is $p\beta_a + (1-p)\beta_b$, and the updated beliefs follow from Bayes' rule. Every term inside Equation (3) is a straight line, so $V(p)$ can be worked out directly. No simulation is needed.

The system then prices probing at its own cost minus the value of the information it buys, and picks the cheapest action:

$$\text{price}(\text{probe}) = 1 + 24p - V(p). \quad (4)$$

Two simple facts fix the band at both ends. First, $V(p) \geq 0$ always, because the smallest of several straight lines is a concave function, and information can never make a decision worse. Second, $V(0) = V(1) = 0$, because when we are already certain, no answer can change what we do. So the band cannot spread across the whole range, and it cannot be widened by tuning.

With our fixed costs and measured bite rates, the calculated result is:

$$\text{pass if } p < 0.065, \quad \text{probe if } 0.065 \leq p < 0.879, \quad \text{divert if } p \geq 0.879. \quad (5)$$

One detail in Equation (5) is worth noticing. Offering a probe pushes the divert point up , from 0.816 to 0.879. A system that can probe is more patient with an honest user than one that cannot. We did not aim for this. It falls out of the arithmetic.

3.5 Measuring the Probe, and Tools Used

The calculation above needs one measured input: how often each kind of visitor takes the bait. We measure this in a separate round, run after the probes were built and locked before the evaluation. For our main probe, 138 of 183 hostile sessions took the bait, and 0 of 74 harmless sessions did. This gives $\beta_a = 0.753$ and $\beta_b = 0.0067$. We do not set β_b to zero, even though no harmless session took the bait, because that would make the arithmetic infinite and would assume away the safety property we are trying to measure.

As the conference guidelines require, we state which tools were used and where. All detection code, features, decision rule and experiments are our own, written in Python, and no language model is used anywhere in the request path. Language models were used in three places, and all three affect results reported here.

1. **As simulated attackers.** We drove three local open models (Llama 3.2 1B, Llama 3.2 3B and Qwen 2.5 7B) as automatic attackers. Each model read the replies and chose its own next request, and none was told that probes existed. This tests our assumption about how curious an attacker is.
2. **As a content generator for the fake site.** A local language model was used to write content for the fake copy of the site in one comparison, to check that our consistency layer does not depend on how the content is produced.
3. **For adversarial testing.** A language model agent was used to attack our own running system and look for mistakes. It found several flaws in the fake site, which we fixed and report in Section 5.

3.6 Experimental Setup

We compare three systems. They differ only in one setting, and all use the same locked model.

- **B1**, a signature firewall using fixed patterns.
- **B2**, our detector without probes. It can only pass or divert.
- **B4**, the full system, with probes and the fake site.

Each system was run over 99 separate rounds of generated traffic. Each round has 120 attack sessions and 80 harmless ones. That gives 11,880 attack sessions and 7,920 harmless sessions per system. Within one round, all three systems see

Table 2. The randomised comparison inside the full system. Both groups reached the probing band under identical rules. Only the probe differs.

group	sessions caught	rate
probe sent	10,643 10,112	0.950
probe withheld	1,237 1,089	0.880

Table 3. Attack recall and harm to normal users, over 99 rounds. Higher recall is better. For diverted harmless sessions, zero is best.

system	attack recall (95% CI)	spread	harmless	diverted
B1 signature firewall	0.366 [0.358, 0.375]	0.020	0 / 7,920	
B2 detector only	0.889 [0.883, 0.894]	0.024	4 / 7,920	
B4 full system	0.943 [0.939, 0.947]	0.018	0 / 7,920	

exactly the same traffic, so every attack session can be compared directly across systems. We fixed the main comparison before running anything.

Throughout this paper, *diverted* means the system decided the visitor was hostile. It is counted the same way in every system, so the numbers are comparable.

4 Results

4.1 What the Probe Itself Causes

Comparing B2 and B4 compares two whole systems, which differ in more than the probe. To measure the probe alone, we withhold it from about one session in ten among those that reach the probing band. The choice is random. Both groups sit at the same belief, under the same rules. The only difference is whether a probe was sent. Table 2 gives the outcome.

Table 2 shows a difference of **7.0 percentage points**. The 95% confidence interval is [5.2, 8.8], and Fisher’s exact test gives $p = 3.4 \times 10^{-19}$. Because the two groups were separated at random at the same belief, this difference is caused by the probe. It is not a comparison of two different designs.

We also checked that the random split was fair. The largest difference in composition between the two groups across the five attack types is 1.2 percentage points. A chi-square test gives 2.58 on four degrees of freedom, well below the 9.49 that would signal a problem. This matters, because some attack types are much easier to catch than others. An unfair split would have measured the mixture rather than the probe.

4.2 Comparison With Baselines

Table 3 compares the three systems.

Table 4. OWASP ModSecurity Core Rule Set on the same traffic. Perfect recall is only reached at a setting no real site could use.

setting	attack recall (95% CI)	harmless sessions blocked
1 (default)	0.353 [0.305, 0.404]	0 / 240
2	0.544 [0.493, 0.595]	0 / 240
3	0.544 [0.493, 0.595]	0 / 240
4 (maximum)	1.000 [0.989, 1.000]	72 / 240 (30%)

Two columns in Table 3 point in opposite directions. For recall, higher is better. For harmless sessions diverted, zero is best. B4 is best on both.

The recall ranges for B2 and B4 do not overlap. Comparing the same sessions in both systems, 842 attacks were caught by B4 but missed by B2, while 197 were caught by B2 but missed by B4. A paired test gives $p = 1.9 \times 10^{-95}$. B4 was ahead in all 99 rounds, which rules out a lucky result.

The 197 sessions that went the other way are not noise, and we do not present them as such. They are delays. Their belief landed between 0.816 and 0.879, where B2 diverts at once and B4 first asks a question. Some of those sessions end before the question is answered. That is the real cost of having a third action.

The two zeros in the last column deserve an explanation. B1’s zero is unsurprising, because its patterns need suspicious syntax that normal traffic does not contain. B4’s zero is more interesting. B4 is not merely as safe as B2. It is safer. All four harmless sessions that B2 wrongly diverted reached the probing band in B4. Each was probed instead of diverted. None took the bait, because they were genuinely harmless. That silence kept their belief below 0.879, while B2 had already acted at 0.816.

4.3 Comparison With a Real Firewall

B1 is a firewall we wrote, and a reader may reasonably distrust it. So we replayed the same traffic through the OWASP ModSecurity Core Rule Set, which many real websites use. Table 4 gives the result at each of its four settings.

Table 4 makes two points. At its default setting the real rule set scores 0.353, close to our B1 at 0.366, so B1 is not a weakened stand-in. And the limit is not a matter of tuning. The rule set only reaches perfect recall by blocking 30% of legitimate visitors, which no real site could accept. At every setting that leaves normal users alone, it stops near 0.54. Our full system reaches 0.943 while diverting nobody.

One point favours the rule set, and we state it plainly. It is judged on blocking any single request, while our systems are judged on identifying the whole session. Blocking one request is the easier task.

Table 5. Recall by attack type. The gain is concentrated in one type: the case where the detector cannot decide on its own.

attack type	sessions	B2	B4	gain
Obfuscated SQL injection	3,960	0.898	0.921	+0.022
Slow SQL injection	1,980	1.000	1.000	0.000
Record walking via API	1,980	0.994	0.993	-0.001
Password spraying	1,980	1.000	1.000	0.000
Record walking via web pages	1,980	0.540	0.823	+0.282

4.4 Where the Probe Helps

Our theory predicts something specific. The probe should help where the detector is genuinely unsure, and nowhere else. Table 5 tests this.

Table 5 shows the whole gain sitting in one row. Walking through records using the normal web pages improves from 0.540 to 0.823. Everywhere else the probe changes almost nothing. This is exactly the case a signature cannot see and the detector cannot settle, which is where our theory says information is worth buying.

One attack type cannot be helped at all, and it is half of what the full system still misses. Obfuscated SQL injection never takes the bait, even though nine in ten of those sessions were shown one. The reason is in the attacker, not the defence. Those attackers send payloads and never read the replies, so a probe placed in a reply cannot reach them.

5 Discussion

Our results support the mechanism, not just a headline number. If the probe had raised recall everywhere, that would suggest a weak detector rather than a correct calculation. Instead the gain sits almost entirely in the one case the detector cannot settle. That is what the pricing predicts. Information is worth buying exactly where the decision is in doubt, and worth nothing elsewhere.

The safety result has the same shape. Raising the divert point from 0.816 to 0.879 was not a safety feature we added. It came out of the arithmetic. Its effect was to recover every mistake the simpler detector made.

Where our approach does not win. Setting the band edges by hand beats calculating them, if expected cost is the only measure. We report this rather than hide it. The reason is that the belief takes only about two distinct values in our system, so the extra precision of a calculated edge does no real work. Every setting that beats our calculated edges does so by diverting harmless users. Among settings that divert nobody, our calculated edges are the best available. That is a smaller claim than we first hoped to make, and a more useful one.

Adversarial testing of the fake site. We used a language model agent to attack our own system, as described in Section 3.5. It found four ways to tell the fake site from the real one, each found by attacking the fix for the one before. The same page could return different values after a move. A list page could name a person differently from their own profile page. A record that did not exist before a move could exist after it. Later, a human volunteer found a fifth: two profiles carried the same name but different details. All five are fixed. We report them because the lesson generalises. Checking that a fake site agrees with itself is not the same as checking that it agrees with what the attacker already saw.

Limitations. Four, in order of importance. First, all our traffic is generated, not real. Every rate we give describes our own traffic, and would need measuring again on real traffic. The calculation and the randomised design do not depend on the traffic being real, but the numbers do. Second, we tune and test on one application. We placed the locked detector in front of a second, very different application and the general features still worked, but the probes themselves would need rewriting. Third, only three people have tested the fake site so far, so whether a human attacker senses something is wrong is still barely measured. Fourth, the bite rate depends on how curious we assumed attackers are. We reduced this risk by testing against language model agents that were told nothing about the probes, but an agent is still not a person.

6 Conclusion

Passive defences force a choice between acting too early and acting too late. That choice can be avoided. A defender can create evidence by placing a small invisible probe in a reply, and the decision of when to do so can be calculated rather than guessed. Priced by the value of the information it buys, probing is the cheapest action over a band that does not exist under ordinary cost accounting. The probe therefore cannot be a tuned threshold. Our measurements support the mechanism. The gain appears only where the detector is genuinely unsure, a randomised comparison shows the probe caused it, and no legitimate session was ever diverted. What remains is to test the idea against real traffic and real attackers.

References

1. Almeshekah, M.H., Spafford, E.H.: Planning and integrating deception into computer security defenses. In: Proceedings of the 2014 New Security Paradigms Workshop. p. 127–138. NSPW '14, Association for Computing Machinery, New York, NY, USA (2014). <https://doi.org/10.1145/2683467.2683482>, <https://doi.org/10.1145/2683467.2683482>
2. Amouei, M., Rezvani, M., Fateh, M.: Rat: Reinforcement-learning-driven and adaptive testing for vulnerability discovery in web application firewalls. *IEEE Transactions on Dependable and Secure Computing* **19**(5), 3371–3386 (Sep 2022). <https://doi.org/10.1109/TDSC.2021.3095417>

3. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: *Proceedings of the 31st USENIX Security Symposium*. pp. 3971–3988. Boston, MA, USA (2022)
4. Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. *ACM Trans. Inf. Syst. Secur.* **3**(3), 186–205 (Aug 2000). <https://doi.org/10.1145/357830.357849>, <https://doi.org/10.1145/357830.357849>
5. Bowen, B.M., Hershkop, S., Keromytis, A.D., Stolfo, S.J.: Baiting inside attackers using decoy documents. In: Chen, Y., Dimitriou, T.D., Zhou, J. (eds.) *Security and Privacy in Communication Networks*. pp. 51–70. Springer Berlin Heidelberg, Berlin, Heidelberg (2009)
6. Bridges, R.A., Mitchell, T.R., Muñoz, M., Henriksson, T.: Sok: Honeypots & llms, more than the sum of their parts? In: *2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P)*. pp. 910–928 (July 2026). <https://doi.org/10.1109/EuroSP68448.2026.00063>
7. Elkan, C.: The foundations of cost-sensitive learning. In: *Proceedings of the 17th International Joint Conference on Artificial Intelligence - Volume 2*. p. 973–978. IJCAI'01, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (2001)
8. Ferguson-Walter, K.J., Major, M.M., Johnson, C.K., Muhleman, D.H.: Examining the efficacy of decoy-based and psychological cyber deception. In: *Proceedings of the 30th USENIX Security Symposium* (2021)
9. Howard, R.A.: Information value theory. *IEEE Transactions on Systems Science and Cybernetics* **2**(1), 22–26 (Aug 1966). <https://doi.org/10.1109/TSSC.1966.300074>
10. Iliou, C., Kostoulas, T., Tsikrika, T., Katos, V., Vrochidis, S., Kompatsiaris, Y.: Towards a framework for detecting advanced web bots. In: *Proceedings of the 14th International Conference on Availability, Reliability and Security. ARES '19*, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3339252.3339267>, <https://doi.org/10.1145/3339252.3339267>
11. Juels, A., Rivest, R.L.: Honeywords: making password-cracking detectable. In: *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security*. p. 145–160. CCS '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2508859.2516671>, <https://doi.org/10.1145/2508859.2516671>
12. Kahlhofer, M., Achleitner, S., Rass, S., Mayrhofer, R.: Honeyquest: Rapidly measuring the enticingness of cyber deception techniques with code-based questionnaires. In: *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. p. 317–336. RAID '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3678890.3678897>, <https://doi.org/10.1145/3678890.3678897>
13. Kahlhofer, M., Golinelli, M., Rass, S.: Koney: A cyber deception orchestration framework for kubernetes. In: *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. pp. 690–702 (June 2025). <https://doi.org/10.1109/EuroSPW67616.2025.00084>
14. Kahlhofer, M., Rass, S.: Application layer cyber deception without developer interaction. In: *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. pp. 416–429 (July 2024). <https://doi.org/10.1109/EuroSPW61312.2024.00053>

15. Kruegel, C., Vigna, G.: Anomaly detection of web-based attacks. In: Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS '03). pp. 251–261 (2003). <https://doi.org/10.1145/948109.948144>
16. Pawlick, J., Colbert, E., Zhu, Q.: A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy. *ACM Comput. Surv.* **52**(4) (Aug 2019). <https://doi.org/10.1145/3337772>, <https://doi.org/10.1145/3337772>
17. Pawlick, J., Colbert, E., Zhu, Q.: Modeling and analysis of leaky deception using signaling games with evidence. *IEEE Transactions on Information Forensics and Security* **14**(7), 1871–1886 (July 2019). <https://doi.org/10.1109/TIFS.2018.2886472>
18. Provos, N.: A virtual honeypot framework. In: Proceedings of the 13th USENIX Security Symposium. pp. 1–14. San Diego, CA, USA (2004)
19. Robertson, W., Vigna, G., Kruegel, C., Kemmerer, R.A., et al.: Using generalization and characterization techniques in the anomaly-based detection of web attacks. In: NDSS (2006)
20. Sladić, M., Valeros, V., Catania, C., Garcia, S.: Llm in the shell: Generative honeypots. In: 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). pp. 430–435 (July 2024). <https://doi.org/10.1109/EuroSPW61312.2024.00054>
21. Sommer, R., Paxson, V.: Outside the closed world: On using machine learning for network intrusion detection. In: 2010 IEEE Symposium on Security and Privacy. pp. 305–316 (May 2010). <https://doi.org/10.1109/SP.2010.25>
22. Srinivasa, S., Pedersen, J.M., Vasilomanolakis, E.: Towards systematic honeypot fingerprinting. In: 13th International Conference on Security of Information and Networks. SIN 2020, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3433174.3433599>, <https://doi.org/10.1145/3433174.3433599>
23. Tekerek, A.: A novel architecture for web-based attack detection using convolutional neural network. *Computers & Security* **100**, 102096 (2021). <https://doi.org/10.1016/j.cose.2020.102096>, <https://www.sciencedirect.com/science/article/pii/S0167404820303692>
24. Vero, M., Kaczmarczyk, F., Petrov, I., Shumailov, I., Hayes, J., Heinen, N., Fan, T., Invernizzi, L., Vechev, M.: Honeyval: A comprehensive evaluation framework for llm-powered http honeypots (2026), <https://arxiv.org/abs/2605.29963>
25. Vetterl, A., Clayton, R.: Bitter harvest: systematically fingerprinting low- and medium-interaction honeypots at internet scale. In: Proceedings of the 12th USENIX Conference on Offensive Technologies. p. 9. WOOT'18, USENIX Association, USA (2018)
26. Yuill, J., Zappe, M., Denning, D., Feer, F.: Honeyfiles: deceptive files for intrusion detection. In: Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop, 2004. pp. 116–122 (June 2004). <https://doi.org/10.1109/IAW.2004.1437806>
27. Zhu, M., Anwar, A.H., Wan, Z., Cho, J.H., Kamhoua, C.A., Singh, M.P.: A survey of defensive deception: Approaches using game theory and machine learning. *IEEE Communications Surveys & Tutorials* **23**(4), 2460–2493 (Fourthquarter 2021). <https://doi.org/10.1109/COMST.2021.3102874>